The hardening agent

What runs when a setter presses **Run hardening**, from the empty queue to the score on the page. The model is optional. Nothing it says is trusted until the same sandbox that judges contestants has run it.

What the agent is for

A setter writes a statement, a reference solution, and a set of tests. The dangerous case is a suite that looks finished and is not. A small bug in the reference, the kind a contestant writes without noticing, still passes every sample. The agent exists to find that hole and to propose the smallest tests that close it.

It does not publish the problem, edit the tests, change the limits, or touch a score. Those actions are denied. What it may do is read the draft, run programs in the sandbox, and leave proposals in a `held` state. The suite changes only when a setter approves a proposal. Approval appends hidden tests, or a stored wrong solution, onto the current draft. A separate publish check then decides whether that draft may go live.

If no model is configured, the same pipeline still runs. The note on the run says the deterministic suite is what produced the result. The score, the mutants, and the proposals do not require a model.

What has to exist first

A run is refused until the draft has a reference solution and at least one test. The setter's page saves unsaved edits before it queues the run, so the worker sees the statement, samples, and zip that are on screen.

Each test is an input and the output the setter claims is correct. Files in a zip are paired by name: `sample1.in` with `sample1.out`. A name that starts with `sample` is a visible sample. Every other pair is hidden. The agent treats a visible sample differently from a hidden test: a mutant killed by a sample is called trivial and is left out of the score.

An input spec is a small grammar for legal stdin, for example `y int 1..9999`. If the setter wrote one and it parses, that spec is used. If it does not parse, it is ignored and the run says so. If there is no spec, the worker tries to draft one from the constraints in the statement and the shape of the tests already uploaded. A draft is kept only when every current test validates against it. A wrong guess is thrown away. The page shows the guess as a drafted spec. The setter can copy it into the spec field.

With neither a written spec nor a draft, boundary, random, and maximum-size generation are skipped. The run then makes a few small edits of the tests the reference already accepts, and it will keep a model input only when the reference answers it.

From the button to a worker

1. The setter's browser sends `POST /api/problem-versions/:id/hardening-runs`. Only a setter or an admin is allowed. The server checks the reference and the tests, inserts a row with status `queued`, and appends an audit row `hardening.start`.
2. One run per draft is the rule. A second click while one is queued or running gets HTTP 409.
3. The server does not run the suite itself. It appends the run id to the Redis stream `agent:harden`. If Redis refuses the append, the row is marked failed and the setter sees that the queue is unavailable.
4. The agent process is a separate worker. It joins the consumer group `hardeners` and blocks on that stream. The hosted worker also writes a heartbeat, once a minute, into `service_status`. The page's AI pill reads that heartbeat when it is less than three minutes old, so the pill describes the worker that has the model key, not the web server.
5. The worker claims the row by flipping `queued` to `running` and storing a random claim token. A second worker that reads the same message finds the row already claimed and skips it. While the run is alive the worker refreshes `updatedAt` every 20 seconds, and again at every stage.
6. Results are written only while that claim still matches. If the claim has been cleared, the worker deletes the mutations and proposals it just inserted rather than leave a second copy.
7. A message whose owner has not acknowledged it for five minutes is reclaimed with `XAUTOCLAIM` and run again. A run that stops updating for three minutes, or that has been running for ten, is marked timed out. Ten minutes is also checked between stages, so a stuck sandbox does not hold the claim past the cap.

Every sandbox batch and every model call is an audit row with action `agent.tool`. Long source is stored as its first 2 KB plus a SHA-256 of the whole text. The audit collection is append-only and lives on its own connection, so a rollback of the draft does not erase the record of the call.

The pipeline

The engine is one function, `runHardening`. The page groups its stages into five labels. The worker's `stage` field is finer. This is the order a run actually walks.

On the page	Stage field	What happens
Baseline	`baseline`	Reference runs on the current tests.
Mutate	`mutants`	Up to 40 single-edit mutants, then each one on those same tests.
Attack	`ai plan`, `attacks`	New inputs, then each surviving mutant on those inputs.
Verify	`reference on attacks`, `ai feedback`	The reference must answer an input before it counts. A second model call, only if mutants remain.
Propose	`proposals`, `performance`	A small cover of tests, then a max-size check.

Baseline

The reference is compiled once and run on every current test in one sandbox batch. If it does not compile, the run fails immediately: there is nothing to mutate. Each test is then compared with the output the setter stored. A mismatch does not abort the run. It adds a note that the reference fails some of the current tests, and the score should not be trusted until those are fixed. Samples are the tests marked visible. If none are marked, the first test stands in as the sample.

A program "runs cleanly" when the process exits 0 and the timer did not kill it. Two outputs match when the judge's comparison says they do. A mutant is caught when it crashes, times out, or prints something else.

Mutants

Mutants are not random edits of the whole file. The engine scans the reference for a fixed list of operators and applies one edit per mutant. Comments, string literals, and character literals are masked off first, so a word inside a comment is never flipped. Generic types such as `vector<int>` are not treated as comparisons.

Operator	The one edit
Relational	`<` with `<=`, `>` with `>=`, `==` with `!=`, and the same for `===`.
Arithmetic	`+` with `-`, `*` with `/`. In Python, `/` becomes `//` and `//` becomes `*`. An operator is touched only when it sits between two operands.
Literal	An integer constant becomes one greater, and, unless it is zero, one smaller. Leading zeros are left alone.
Range	Python `range(n)` becomes `range(n - 1)` and `range(1, n)`.
Logical	`and` with `or`, or `&&` with `||`.
Min / max	The call `min(` becomes `max(`, and the reverse.
Break	`break` becomes `pass` in Python and an empty statement in the other languages.
Overflow	In C and C++, `long long` becomes `int`.

Literals are the noisiest operator, so the generator does not take the first 40 edits in file order. It takes one edit from each operator, then a second round, until it has 40 mutants or the operators run out. Two edits that produce the same source are kept once. Ids are `m1`, `m2`, and so on. The page strips the leading `m` on the tiles.

Each mutant is then run on the current tests.

Status	Meaning
`compile_error`	The mutant does not compile. It is not a bug the suite should be praised for catching.
`trivial`	A deterministic mutant already fails a sample. Left out of the score.
`killed`	It fails a hidden test, or it fails an attack later in the run.
`survived`	It matches the reference on every input tried so far.
`equivalent`	It survived the whole suite and at least ten random inputs. Treated as a likely no-op edit, not as a hole.

A model-written wrong solution is stored as a mutant too, with source `model` and an id `ai1`, `ai2`, ... A slow model solution is marked `slow`. Model mutants are replayed on the current tests the same way. They do not enter the numeric score. The score is computed only from deterministic mutants of the reference.

Attack inputs

New inputs are generated only when they are not already in the suite. Each candidate is checked before it is kept.

- With a spec, the input must validate. An illegal input is dropped and the reason is kept in the notes.
- Without a spec, a model input or a shape edit longer than 20 000 characters is dropped. Shorter ones are labelled "kept only if the reference answers them".
- The reference is run on the survivors. An input is dropped if the reference crashes, times out, prints nothing, or prints more than 64 KB. The reference's stdout becomes the expected output. The setter's old tests are not rewritten.

When a spec exists, three deterministic families are built from it. The random number generator is seeded from the run id, so the same run reconstructs the same inputs.

- **Boundary.** One input for each class the spec allows: minimum size, a single element, maximum values, all values equal, sorted, reverse sorted, zeros, and negatives. A class the spec cannot express, such as negatives on a year from 1 to 9999, produces nothing.
- **Random.** Thirty random valid inputs, skipping duplicates.
- **Max scale.** The largest valid input that still fits under 1 MB, found by halving the size cap until the generator succeeds.

Without a spec, shape edits stand in for that list, at most twelve: move the largest integer on a line to the end, shrink a counted list to one element, set the last value to 0 or to -1, add one to the last integer, or drop the last line. Anything the reference cannot answer is still dropped.

Every mutant that is still `survived` is then run on the accepted attack inputs. An input that makes a mutant differ from the reference is a kill. A mutant that agrees with the reference on a random input increments a counter. After the attacks, a deterministic mutant with no kill and at least ten of those random survivals is marked `equivalent`.

The model, when there is one

The planner is allowed two calls in a run, and the second happens only when some mutant is still unkilled.

The first call is the plan. The prompt contains the statement (clipped), the spec or the word “none”, the reference, the first few current inputs, and up to twelve surviving mutants shown as a one-line diff. It asks for wrong solutions a contestant might write, edge-case inputs, and slow solutions that a maximum-size test should catch.

The second call is feedback. It receives the mutants that survived every test and every attack so far, and it asks for inputs that make each one print a different answer.

The reply must be one JSON object. Prose and code fences are tolerated: the parser takes the first `{` through the last `}`. The object has a single useful field.

```
{"attacks": [
  {"type": "edge_case", "description": "year 100", "input": "100\n"},
  {"type": "wrong_solution", "description": "forgets the century", "language": "python", "code": "..."},
  {"type": "performance", "description": "slow on the largest year", "language": "python", "code": "..."}
]}
```

Each attack needs a `code`, an `input`, or a `generator`. A generator is Python 3 that prints one stdin; it is itself run in the sandbox, and its stdout is the input, provided that stdout is non-empty and at most 1 MB. At most twelve attacks are read from one reply, and the prompt asks for eight. A field the schema does not allow is dropped, not silently kept.

A `wrong_solution` or `performance` attack becomes a model mutant and is replayed on the current tests. An `edge_case` becomes another attack input and goes through the same validator and the same reference run as a boundary input. Feedback attacks are a second wave: validate, ask the reference for the output, then strike the mutants that are still alive.

The call is skipped, and the run continues, when any of these is true.

- `AGENT_PROVIDER` is unset, `none`, or `fake`, or the model name or the key is missing. Status `DISABLED`.
- The provider answers 429, including after one retry. Status `RATE_LIMITED`. For Gemini, a limited key is cooled for 45 seconds and the next key in the pool is tried. A rejected key is not tried again in that process.
- This run has spent its token cap (20 000 by default) or the month-to-date audit total has reached the monthly cap (200 000 by default). Status `BUDGET_EXCEEDED`. The denied call is audited with the reason.
- The endpoint times out at 60 seconds, is unreachable, or returns anything other than usable JSON. Status `ERROR`.

The provider is selected only from the environment: Gemini, Anthropic, or an OpenAI-compatible base URL. Nothing is assumed when the variables are empty. Gemini may list several keys; the pool walks them. Token totals on the page are the provider’s reported counts for the calls that returned.

What the model is not allowed to do

The prompt tells the model it cannot publish, edit tests, or change limits. The code does not rely on the model obeying that sentence. A tool name anywhere in the reply is checked against a fixed table. Unknown names are denied.

Allow	Hold	Deny
`read_problem`	`propose_test`	`publish`
`run_in_sandbox`	`propose_wrong_solution`	`edit_tests`, `delete_tests`
		`change_limits`, `change_score`, `disqualify`

A denied tool is dropped from the reply and written to the audit as `deny`. The rest of a well-formed attack list is still used. Hold is the state of a proposal after a successful run: it is visible, and it has no effect on the draft.

Sandbox calls are the other allowed path, and they are not made by the model. The worker calls the judge's batch runner: one compile, then every input, inside the same limits as a contest submission for that language. The default is 2 seconds and 256 MB, or 3 seconds and 512 MB for Java, unless the draft sets its own limits. The container has no network, a read-only root filesystem, a CPU cap, and a kill on timeout. A mutant cannot be "caught" by the model asserting that it failed. It is caught only when that container's stdout, exit code, or timer disagrees with the reference.

Proposals

The run now has a map from attack inputs to the mutants each input kills. The proposal step is a greedy cover, not a dump of every kill.

While some killed-but-not-yet-covered mutant remains, and fewer than ten test proposals have been emitted, the engine picks the input that kills the most of those still-open mutants. A tie goes to the shorter input. That input becomes one proposal: the stdin, the reference's stdout, the ids of the mutants it covers, and a short reason that names the first mutant and the attack that found the input (`boundary:min`, `random`, `max-scale`, `model:edge_case`, and so on). Evidence on the proposal is the validator result, the reference's run, and one mutant's run, each clipped. Those ten are a cap on the test cover. If mutants remain uncovered, the note says to approve these and run again.

Two further kinds of proposal are added after the cover.

- A model wrong solution that the current tests already kill, or that one of the new tests kills, is proposed as a stored wrong solution. The reason says that keeping it guards later edits of the suite. A model solution that matches the reference on every input is not proposed; the note says it may be correct.
- If a maximum-size input was accepted and the largest current test is smaller than half that size, a performance test is proposed. The reason quotes the reference's time against the limit. Slow solutions the setter already attached, and slow model mutants, are run on that one input. Those the input catches are named on the proposal, and a slow model mutant caught this way is marked killed.

The score

Let the eligible mutants be the deterministic ones whose status is `killed` or `survived`. Compile errors, trivial mutants, and likely-equivalent mutants are out. Model mutants are out.

$$\text{score} = \frac{\text{killed now}}{\text{eligible}} \qquad \text{projected} = \frac{\text{killed now} + \text{survivors a proposal covers}}{\text{eligible}}$$

The score is null when nothing is eligible, which is also the case when no operator applied to the reference. The page prints a dash for that. The projected score is what the same ratio would be after the held test proposals are approved. It is a forecast, not a second measurement: the mutants have already been run.

Beside the score, the metrics record how many attacks of each family were kept, whether each boundary class was generated, and whether any current test already covers that class. The performance block records the max-input size, the largest current test, the reference's time, and whether a performance test was proposed.

Leap year, from an empty draft

This is the draft in `demo/authoring`. The statement says a year from 1 to 9999 is a leap year when it is divisible by 4, except years divisible by 100, except years divisible by 400. The reference is three lines.

```
y = int(input())
leap = y % 4 == 0 and (y % 100 != 0 or y % 400 == 0)
print("Leap" if leap else "Common")
```

The zip has sixteen pairs. The two samples are 2024, which prints `Leap`, and 2023, which prints `Common`. The other fourteen are hidden and, by design, include no year divisible by 100 or by 400. The spec is `y int 1..9999`.

The operators that fire on line 2 are the relational flips (`==` with `!=`), the literals 4, 100, and 400 each moved by one, the three comparisons against zero moved from 0 to 1, and `and` swapped with `or`. That is fourteen mutants, under the cap of forty, so all of them are kept. A mutant that changes only the test on 4, or the `and`, fails 2024 or 2023 and is trivial. A mutant that changes only the 100 or the 400 still agrees with the reference on every year in the zip, so it survives.

Boundary and random generation are allowed to emit 100, 400, 1, and 9999, because the spec says those are legal. The

reference prints `Common` for 100 and `Leap` for 400. Those two inputs kill the century mutants the zip never touched. The cover then keeps the shortest inputs that hit the surviving set. On the demo run those proposals were a test whose input is 400 and whose expected output is `Leap`, and a test whose input is 100 and whose expected output is `Common`, together with model attacks aimed at the same hole. The score stays low until those tests are approved, because the trivial mutants never counted and the century mutants are still `survived`. The projected score moves once the proposals cover them.

Approving writes both tests onto the draft as hidden tests. The publish check is a different button. It runs the reference and demands AC on every test, then runs each stored wrong solution and demands that it fail at least one. A wrong solution that is AC on the whole suite blocks publication with `wrong solution survived`. That is how the century bug, once it has been stored as a wrong solution, cannot ship inside a later edit of the tests.

Approval

Proposals stay `held` until a setter selects them. Approve runs in one transaction against the newest version of that problem.

- Test proposals, including performance tests, are appended as hidden tests. An input that is already in the suite is skipped.
- Wrong-solution proposals are appended to the draft's list of wrong solutions, labelled with the proposal id.
- If the newest version is still `draft` or `blocked`, the rows are pushed onto it and its status returns to `draft`, with a cleared publish report.
- If the newest version is already published, or otherwise not editable, a new draft is inserted with the next version number and the same statement, reference, and limits. The approved rows are added there. The published version is not rewritten.
- Two setters approving the same proposal, or two approvals creating the next draft at once, produce HTTP 409. One of them has to retry. The proposal is marked `approved` only inside the transaction that won.

The audit row `proposal.approve` records who approved, which version the rows landed on, and how many tests and wrong solutions were added. After that, the setter runs the publish check. The agent is not in that path.

How a run ends

Status	Why
`completed`	The pipeline finished. The stage is `done`. Notes may still warn that the reference fails current tests, that the model was unavailable, or that more mutants remain than the ten proposals could cover.
`failed`	The reference does not compile, the version disappeared, or an unexpected error escaped. The message is stored on the run.
`timed_out`	The run passed ten minutes, or the worker went quiet for three minutes, or a queued run sat for ten minutes with no worker.
`cancelled`	The claim disappeared between stages because the run was stopped. Partials already written under that claim are removed if the final write loses the race.

A completed run is a record of one draft at one moment: the metrics, the notes, every mutant and its status, and every held proposal with the evidence that killed its target. Approving proposals does not rewrite that record. The next press of the button is a new run, with a new seed, against whatever the draft looks like after the approval.